

সহজ বাজার খাতা

Market Khata

Complete Technical Product Specification

Platform: Flutter (Android & iOS)

Backend: PHP (Laravel) REST API

Version: MVP 1.0

May 2025

1. App Overview

1.1 Product Summary

সহজ বাজার খাতা is a collaborative office market management app. It replaces paper notebooks and WhatsApp group chaos with a structured but flexible digital system for managing daily office and personal shopping tasks, money balances, and expense records.

Core Promise

বাজারের তালিকা, খরচ, আর টাকা কার কাছে কত আছে — সব এক জায়গায় সহজভাবে। (Shopping lists, expenses, and who holds how much money — all in one simple place.)

1.2 Target Users & Roles

The app has exactly 3 roles — simple, clear, real-office-friendly.

Role	Bangla	Who Uses This	Core Responsibility
admin	অ্যাডমিন	System administrator	Create wallets, assign owners, manage users, correct mistakes, monthly close
owner	মালিক	CEO, CTO, Accounts Officer, any designated wallet manager	Manage assigned wallet(s) — balance, money entries, reports, edit bazar lists
assistant	সহকারী	Office assistant(s)	Shop for any wallet, enter prices, upload receipts, add/update items

Real Office Setup Example

Step 1 — Admin creates wallets: "Office", "CEO Personal", "CTO Personal"
Step 2 — Admin assigns owners: CEO → "CEO Personal", Accounts Officer → "Office", CTO → "CTO Personal"
Step 3 — All assistants automatically see every wallet and can shop for any of them.
Result: CEO only sees his wallet. Accounts Officer only sees Office wallet. Assistants see all.

1.3 What This App Is NOT

- Not an ERP or strict accounting software
- Not a product inventory system
- Not a barcode / POS system
- Not a payroll system

2. Core Concepts

2.1 Wallet (ওয়ালেট)

A Wallet replaces the older "Khata" concept. Each wallet represents a money pool for a specific purpose or person.

Wallet Name	Type	Typical Managers
Office Wallet	Shared	Accounts Officer 1, Accounts Officer 2
CEO Wallet	Personal	CEO + assigned assistant
CTO Wallet	Personal	CTO + assigned assistant
Office Lunch Wallet	Shared	Accounts team
Event Wallet	Temporary	Event coordinator

Key rules:

- A wallet can have one or more managers (e.g. two Accounts Officers can both manage Office Wallet)
- Any assistant can view and create expenses for a wallet they are permitted on
- Wallet balance is NEVER stored — it is always dynamically calculated

2.2 Bazar (বাজার Session — Optional Grouping)

A Bazar is an optional grouping mechanism. Assistants are NOT required to create a bazar before entering expenses.

Two valid workflows

(A) Quick Expense: Assistant goes to shop, comes back, adds expense directly to wallet. (B) Planned Bazar: Someone creates a list with items, assistant picks it up, buys items, closes it.

A Bazar can be:

- Open (anyone available can take it)
- Requested (CEO says "Rahim will do this bazar" — optional assignment)

2.3 Market Item

Each item inside a bazar requires only a name. Everything else is optional.

Field	Required?	Description
name	YES	Item name in Bangla or English
quantity	No	Amount to buy
unit	No	packet, kg, piece, etc.
attributes	No	Brand, type, notes e.g. "Aarong full cream"

Field	Required?	Description
note	No	Free text instruction
actual_quantity	No	Updated during/after shopping
price	No	Final purchase price — triggers expense when saved
status	Auto	pending / done / not_found / cancelled

Automatic Expense Rule

When an item is saved with price > 0, the item status becomes "done" automatically and the price is counted as an expense for that wallet. No separate expense entry needed.

2.4 Item Status Flow

Status	Bangla	Meaning
pending	বাকি	Not yet purchased
done	কেনা হয়েছে	Price entered → auto-counted as expense
not_found	পাওয়া যায়নি	Could not buy
cancelled	লাগবে না	Removed from list

Note: No "partial" or "extra" status needed. Quantity can be updated freely. Activity log records all changes.

2.5 Money Entry Types

Type	Bangla Label	Balance Effect
money_received	টাকা নিলাম	+ (assistant received money from wallet)
money_returned	টাকা ফেরত দিলাম	- (assistant returned unused money)
adjustment	সমন্বয়	+/- (admin correction, always needs note)

Note: Normal item purchase expense does NOT need a money_entry. It is auto-calculated from item prices.

3. Balance Calculation Logic

3.1 Core Formula

```
Balance (per wallet, per assistant) =
  SUM(money_received)
  - SUM(item_prices where status = 'done')
  - SUM(direct_expenses)      ← expenses entered without a bazar
  - SUM(money_returned)
  +/- SUM(adjustments)
```

IMPORTANT: Balance is NEVER stored in any table.
It is ALWAYS calculated fresh from source entries.

3.2 Why Dynamic Calculation?

During shopping, entries can be wrong. Example: assistant enters "30 eggs = 200 BDT" but after returning to office realizes it was "28 eggs = 180 BDT." Since balance is recalculated from entries, editing the item automatically corrects the balance. No manual adjustment entry needed.

3.3 In-Progress vs Confirmed Balance

The app displays two balance figures when a bazar is still open:

Balance Type	Source	Meaning
Confirmed Balance	Closed bazars + direct expenses + money entries	Settled, reliable figure
In Progress	Open/active bazars	Tentative, may change
Estimated Balance	Confirmed + In Progress combined	Full picture

UI Display Example

CEO Wallet — Rahim Confirmed Balance: +৳ 3,200 In Progress: -৳ 850 (Rahim is currently shopping) Estimated Balance: +৳ 2,350

3.4 Balance Labels (Human Friendly)

Condition	Display Label
Balance > 0	Rahim-এর হাতে আছে ৳ X (Assistant has money remaining)
Balance < 0	Rahim পাবে ৳ X (Assistant is owed money)
Balance = 0	হিসাব মিলে গেছে (Settled)

4. Edit Rules & Audit Safety

4.1 Three-Layer Edit Control

Layer	Trigger	Effect
Edit Window	Bazar still open (চলতেছে)	All items freely editable
Soft Lock	Bazar closed (বন্ধ)	Only Admin can edit; every edit is logged
Hard Lock	Monthly period closed by Accounts	Nobody can edit; permanent

Money entries (money_received, money_returned) can be edited only on the same day they were created. After that, a correction requires an adjustment entry with mandatory note.

4.2 Monthly Closing (হিসাব বন্ধ)

At end of each month, the Accounts Officer performs a monthly close:

1. Accounts Officer reviews all entries for the month
2. Confirms balance with assistant / wallet owner
3. Clicks "হিসাব বন্ধ করুন" in the app
4. System creates a wallet_snapshot record with closing balance
5. All entries for that period become permanently locked (is_locked = true)

```
wallet_snapshots table
├── id                UUID
├── wallet_id         FK
├── assistant_id      FK (nullable – per-assistant snapshot)
├── period_month      YYYY-MM
├── opening_balance   DECIMAL
├── closing_balance   DECIMAL
├── snapshot_hash     VARCHAR(64) ← SHA-256 tamper detection
├── closed_by         FK users
├── closed_at         TIMESTAMP
└── notes             TEXT
```

4.3 Balance Calculation After Monthly Close

```
Current Balance =
  Last Closed Snapshot.closing_balance
  + SUM(entries since snapshot.closed_at)

This limits recalculation to current month only → fast query.
```

4.4 Tamper Detection

On every balance page load, the system re-hashes the locked period entries and compares with stored hash. If mismatch is detected:

- Admin receives an alert
- Affected balance display shows a warning badge
- System logs the discrepancy in audit_alerts table

5. Database Schema

5.1 users

users		
id	UUID	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	UNIQUE
email	VARCHAR(150)	NULLABLE, UNIQUE
role	ENUM	assistant owner admin
password_hash	VARCHAR(255)	
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

5.2 wallets

wallets		
id	UUID	PRIMARY KEY
name	VARCHAR(150)	NOT NULL
type	ENUM	shared personal event other
is_active	BOOLEAN	DEFAULT TRUE
created_by	FK users	
created_at	TIMESTAMP	
wallet_members (pivot)		
id	UUID	
wallet_id	FK wallets	
user_id	FK users	
role	ENUM	owner ← only owners are explicitly listed here assistants get access to all wallets by default via their role
wallet_assistant_restrictions (optional – only populate when restriction needed)		
id	UUID	
wallet_id	FK wallets	
assistant_id	FK users	
created_by	FK users	← admin only
created_at	TIMESTAMP	
RULE: If wallet has NO rows in this table → all assistants can shop for it. If wallet HAS rows → only listed assistants can see & shop for it. This is OPTIONAL. Admin populates only when specific restriction is needed.		
created_at	TIMESTAMP	

5.3 bazars (Market Sessions)

bazars		
id	UUID	PRIMARY KEY
wallet_id	FK wallets	NOT NULL
created_by	FK users	NOT NULL
assigned_to	FK users	NULLABLE ← optional specific assignment
title	VARCHAR(200)	NULLABLE

note	TEXT	NULLABLE
status	ENUM	draft open closed cancelled
bazar_date	DATE	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	
closed_at	TIMESTAMP	NULLABLE

5.4 bazar_items

```

bazar_items
-----
id                UUID                PRIMARY KEY
bazar_id          FK bazars                NOT NULL
name              VARCHAR(200)             NOT NULL
raw_text          TEXT                NULLABLE    ← original user-typed text
quantity          DECIMAL                NULLABLE
unit              VARCHAR(50)             NULLABLE
attributes        TEXT                NULLABLE
note              TEXT                NULLABLE
status            ENUM                pending | done | not_found | cancelled
price             DECIMAL                NULLABLE    ← triggers expense when > 0
                                                         latest state = truth; history in

activity_logs
added_by          FK users
created_at        TIMESTAMP
updated_at        TIMESTAMP

INDEX: (bazar_id, status)
INDEX: (bazar_id, price) WHERE price IS NOT NULL

```

5.5 money_entries

```

money_entries
-----
id                UUID                PRIMARY KEY
wallet_id         FK wallets             NOT NULL
assistant_id      FK users                NOT NULL
bazar_id          FK bazars                NULLABLE    ← optional link to a bazar
type              ENUM                money_received | money_returned | adjustment
amount            DECIMAL                NOT NULL    ← always positive; sign by type
note              TEXT                NULLABLE
entry_date        DATE                NOT NULL
created_by        FK users
created_at        TIMESTAMP
updated_at        TIMESTAMP
is_locked         BOOLEAN                DEFAULT FALSE

INDEX: (wallet_id, assistant_id, entry_date)
INDEX: (wallet_id, is_locked)

```

5.6 direct_expenses (formerly quick_expenses)

```

direct_expenses  ← expense entry without a bazar group
-----
id                UUID                PRIMARY KEY
wallet_id         FK wallets             NOT NULL
assistant_id      FK users                NOT NULL
amount            DECIMAL                NOT NULL
note              TEXT                NULLABLE

```


entry_date	DATE	NOT NULL
receipt_url	VARCHAR(500)	NULLABLE
created_by	FK users	
created_at	TIMESTAMP	
is_locked	BOOLEAN	DEFAULT FALSE

5.7 wallet_snapshots

```
wallet_snapshots
```

id	UUID	PRIMARY KEY	
wallet_id	FK wallets	NOT NULL	
assistant_id	FK users	NULLABLE	← null = aggregate snapshot
period_month	CHAR(7)	NOT NULL	← format: YYYY-MM
opening_balance	DECIMAL	NOT NULL	
closing_balance	DECIMAL	NOT NULL	
closed_by	FK users		
closed_at	TIMESTAMP		
notes	TEXT	NULLABLE	

UNIQUE: (wallet_id, assistant_id, period_month)

NOTE: SHA-256 tamper-detection hash is planned for v1.5.
v1 relies on soft/hard locks and activity_log.

5.8 attachments

```
attachments
```

id	UUID		
bazar_id	FK bazars	NULLABLE	
bazar_item_id	FK bazar_items	NULLABLE	← v2: item-specific receipt
money_entry_id	FK	NULLABLE	
direct_expense_id	FK	NULLABLE	
file_url	VARCHAR(500)		
file_type	ENUM	image pdf	
uploaded_by	FK users		
created_at	TIMESTAMP		

5.9 comments (Contextual Notes — NOT a chat system)

Comments are short contextual notes attached to a bazar. They are NOT a messenger. Real communication happens via phone/WhatsApp. Comments exist only for audit context and clarification (e.g. "Aarong পাইনি, local brand নিয়েছি").

```
comments
```

id	UUID
bazar_id	FK bazars
user_id	FK users
message	TEXT
created_at	TIMESTAMP

5.10 activity_logs

```

activity_logs
-----
id                UUID
user_id           FK users
action            VARCHAR(100)   e.g. item.updated, bazar.closed
entity_type       VARCHAR(50)
entity_id         UUID
old_value         JSON          NULLABLE
new_value         JSON          NULLABLE
ip_address        VARCHAR(45)   NULLABLE
created_at        TIMESTAMP

INDEX: (entity_type, entity_id)
INDEX: (user_id, created_at)

```

6. Roles & Permissions

6.1 Role Summary

The 3-Role Mental Model

admin → God mode. Creates structure. No shopping. owner → Manages their wallet(s). Sees only what is assigned to them. assistant → Does the shopping. Sees all wallets unless a bazar is specifically assigned to someone else.

6.2 Wallet Visibility Rules

Who	Which wallets can they see?
admin	ALL wallets
owner	ONLY wallets they are assigned to (via wallet_members table)
assistant	ALL wallets by default

6.3 Bazar Visibility Rules (Important)

Bazar assigned_to field	Who can see the bazar?
NULL (open, no assignment)	All assistants + wallet owners + admin
Specific assistant (e.g. Rahim)	Only Rahim + wallet owners + admin. Other assistants CANNOT see it.

Why this rule?

If CTO specifically says "Rahim will do my shopping today", other assistants should not interfere or see the private items in that bazar. The assignment creates a privacy boundary.

6.4 Permission Matrix

Permission	assistant	owner	admin
See all wallets	YES	NO (own only)	YES
Create bazar (any wallet)	YES	YES (own wallet)	YES
Pick up open (unassigned) bazar	YES	NO	YES
Add/edit items — open bazar	YES (if visible)	YES (own wallet)	YES
Edit items — closed bazar	NO	NO	YES + logged
Add quick expense	YES	NO	YES
Record money_received	NO	YES (own wallet)	YES
Record money_returned	NO	YES (own wallet)	YES
Add adjustment entry	NO	NO	YES + mandatory note
View wallet balance	YES (all wallets)	YES (own wallet)	YES
Perform monthly close	NO	YES (own wallet)	YES
Export reports	NO	YES (own wallet)	YES
Upload receipts	YES	NO	YES
Add comments	YES	YES	YES
Create wallets	NO	NO	YES
Manage users	NO	NO	YES
Assign owners to wallets	NO	NO	YES

7. REST API Specification

7.1 Authentication

```
POST    /api/auth/login
POST    /api/auth/logout
GET     /api/auth/me
```

```
Auth: Bearer Token (Laravel Sanctum recommended)
All endpoints require Authorization: Bearer {token} header
```

7.2 Users

```

GET      /api/users          list (admin only)
POST     /api/users          create user (admin)
GET      /api/users/{id}     get user
PUT      /api/users/{id}     update user
PATCH   /api/users/{id}/deactivate soft delete (admin)

```

7.3 Wallets

```

GET      /api/wallets        list accessible wallets
POST     /api/wallets        create wallet (admin)
GET      /api/wallets/{id}   wallet detail
PUT      /api/wallets/{id}   update wallet
GET      /api/wallets/{id}/balance current balance breakdown
GET      /api/wallets/{id}/members list members
POST     /api/wallets/{id}/members add member
DELETE   /api/wallets/{id}/members/{uid} remove member

```

7.4 Bazars

```

GET      /api/bazars          list (filters: wallet, status, date,
assigned_to)
POST     /api/bazars          create bazar
GET      /api/bazars/{id}     bazar detail with items
PUT      /api/bazars/{id}     update title/note/assignment
POST     /api/bazars/{id}/close close bazar
POST     /api/bazars/{id}/cancel cancel bazar

```

```

Request body POST /api/bazars:
{
  wallet_id:    UUID,
  title:        string (optional),
  note:         string (optional),
  bazar_date:   YYYY-MM-DD,
  assigned_to:  UUID (optional)
}

```

7.5 Bazar Items

```

POST     /api/bazars/{id}/items add item
GET      /api/bazars/{id}/items list items
PUT      /api/items/{id}        update item (qty, price, status, etc.)
DELETE   /api/items/{id}        cancel item (sets status = cancelled)
POST     /api/items/parse       auto-parse raw text list → item array

```

```

POST /api/items/parse request:
{ "raw_text": "গরুর মাংস ২ কেজি
দুধ ২ প্যাকেট Aarong" }

```

```

Response:
[
  { "name": "গরুর মাংস", "quantity": 2, "unit": "কেজি" },
  { "name": "দুধ", "quantity": 2, "unit": "প্যাকেট", "attributes": "Aarong" }
]

```

7.6 Money Entries

```

GET      /api/money-entries      list (filters: wallet, assistant, type,
date)
POST     /api/money-entries      create entry
GET      /api/money-entries/{id} detail
PUT      /api/money-entries/{id} edit (same day only; after that → 403)
DELETE   /api/money-entries/{id} delete (same day only; after that → 403)

POST /api/money-entries request:
{
  wallet_id:      UUID,
  assistant_id:   UUID,
  type:           money_received | money_returned | adjustment,
  amount:         number,
  note:           string (required for adjustment),
  entry_date:     YYYY-MM-DD,
  bazar_id:       UUID (optional)
}

```

7.7 Quick Expenses

```

POST     /api/direct-expenses    add direct expense (no bazar)
GET      /api/direct-expenses    list
PUT      /api/direct-expenses/{id} edit (same day only)
DELETE   /api/direct-expenses/{id} delete (same day only)

POST /api/direct-expenses:
{
  wallet_id:      UUID,
  assistant_id:   UUID,
  amount:         number,
  note:           string,
  entry_date:     YYYY-MM-DD
}

```

7.8 Balance

```

GET      /api/balances          all wallet balances (accounts/admin)
GET      /api/balances/{wallet_id} single wallet balance
GET      /api/balances/{wallet_id}/{assistant_id} per-assistant balance

Response structure:
{
  wallet_id:      UUID,
  assistant_id:   UUID,
  last_snapshot:  { period, closing_balance, closed_at },
  confirmed_balance: number, ← from snapshot + closed bazars
  in_progress_amount: number, ← from open bazars
  estimated_balance: number,
  label:          "হাতে আছে" | "পাওনা আছে" | "হিসাব মিলে গেছে"
}

```

7.9 Monthly Close

```

POST     /api/snapshots/close    perform monthly close
GET      /api/snapshots          list snapshots
GET      /api/snapshots/{id}/verify verify hash integrity

POST /api/snapshots/close:
{

```

```
wallet_id:      UUID,
assistant_id:   UUID (optional - null = aggregate),
period_month:   "YYYY-MM",
notes:         string
}
```

7.10 Attachments & Comments

```
POST    /api/attachments      upload (multipart/form-data)
GET     /api/bazars/{id}/attachments  list attachments
GET     /api/bazars/{id}/comments    list comments
POST    /api/bazars/{id}/comments    add comment
```

7.11 Reports

```
GET     /api/reports/summary      overall summary
GET     /api/reports/monthly      monthly breakdown
GET     /api/reports/wallet       per-wallet report
GET     /api/reports/assistant    per-assistant report
GET     /api/reports/export       export CSV/PDF (query: format=csv|pdf)
```

Common query params:

```
?wallet_id=UUID
&assistant_id=UUID
&from=YYYY-MM-DD
&to=YYYY-MM-DD
&status=open|closed
```

7.12 Notifications

```
GET     /api/notifications      list (paginated)
PATCH  /api/notifications/{id}/read  mark as read
PATCH  /api/notifications/read-all  mark all as read
```

8. Flutter App Architecture

8.1 Folder Structure

```
lib/
├── main.dart
├── app.dart                ← MaterialApp, routing, theme
├── core/
│   ├── constants/
│   │   ├── api_constants.dart
│   │   └── app_colors.dart
│   ├── theme/
│   │   └── app_theme.dart
│   ├── network/
│   │   ├── api_client.dart    ← Dio/http wrapper
│   │   └── interceptors.dart ← auth token injection
│   └── utils/
│       ├── balance_helper.dart ← label logic
│       └── date_helper.dart
```

```

├── features/
│   ├── auth/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   ├── wallet/
│   ├── bazar/
│   ├── items/
│   ├── money/
│   ├── balance/
│   ├── reports/
│   └── notifications/
├── shared/
│   ├── widgets/
│   │   ├── balance_card.dart
│   │   ├── bazar_status_chip.dart
│   │   └── item_tile.dart
│   ├── models/                ← shared data classes
│   └── services/
│       └── notification_service.dart

```

8.2 State Management

Option	Recommended For	Note
Riverpod	Full app (preferred)	Cleaner, testable, scalable
GetX	Fast MVP	Faster to write, less boilerplate
BLoC	Enterprise teams	Verbose but very structured

Recommendation: Use Riverpod for a production app. GetX if timeline is very tight.

8.3 Key Packages

Package	Purpose
dio	HTTP client with interceptors
flutter_riverpod / get	State management
go_router	Navigation
flutter_secure_storage	Store auth token securely
image_picker	Receipt photo upload
intl	Bangla date formatting
shared_preferences	Light local cache
firebase_messaging	Push notifications
fl_chart	Balance/expense charts in reports
pdf	PDF report generation
drift OR isar	Offline-first local DB (v2)

8.4 Local Caching Strategy (MVP)

- Auth token: flutter_secure_storage
- Wallet list + last balance: shared_preferences (TTL 5 min)
- Active bazar items: in-memory while screen open
- Offline full support: Drift/Isar in Version 2 only

9. UI Screens Specification

9.1 Navigation Structure

```
Bottom Navigation (5 tabs):  
1. Home (হোম)  
2. বাজার ← bazar list + create  
3. হিসাব ← balance overview  
4. রিপোর্ট ← reports  
5. More (আরো) ← settings, notifications, logout
```

9.2 Screen List

Screen	Route	Who Can Access
Login	/login	Public
Home Dashboard	/	All
Wallet List	/wallets	All
Wallet Detail	/wallets/:id	Members
Create Bazar	/bazars/create	Assistant, Manager, Admin
Bazar Detail	/bazars/:id	All with wallet access
Item Edit	/items/:id/edit	Assigned + Manager + Admin
Quick Expense	/expenses/quick	Assistant, Admin
Money Entry	/money/create	Accounts, Admin
Balance Overview	/balance	All
Notifications	/notifications	All
Monthly Close	/close	Accounts, Admin
Reports	/reports	Accounts, Admin
User Management	/users	Admin

9.3 Home Screen Sections

6. Quick Action Buttons: নতুন বাজার | টাকা নিলাম | খরচ লিখি | টাকা ফেরত

7. Today's Active Bazars (wallet-wise cards)
8. My Balance Summary (per wallet)
9. Recent notifications (top 3)

9.4 Bazar Detail Screen Sections

10. Header: Wallet name, assigned person, status chip, date
11. Item List: tap item → update qty/price → auto-done
12. Add New Item — two entry modes (see 9.6 New Bazar screen)
13. ★ Frequent Items row — one-tap add for repeated items from previous bazars (e.g. Office Lunch always has same eggs/bread/milk)
14. Expense Summary: মোট খরচ + current balance (confirmed + in-progress)
15. Activity Timeline — chronological story of the bazar (see 9.5)
16. Attachments (receipt photos)
17. Comments (contextual notes only — short, audit-focused)
18. Action buttons: আজকের বাজার শেষ | বাতিল করুন [NOTE: UI label is soft, backend status is closed/cancelled]

9.5 Activity Timeline (Inside Bazar Detail)

This is the story of a bazar — every change, visible in order. It reduces disputes and makes the log human-readable without needing a separate screen.

Activity Timeline Example

10:30 Rahim → "দুধ ২ প্যাকেট" যোগ করেছে 10:32 CEO → দুধ quantity 1 করেছে 11:20 Rahim → Aarong দুধ পাওয়া যায়নি 11:45 Rahim → local দুধ ১ প্যাকেট কিনেছে — ট 65 12:00 Rahim → বাজার শেষ করেছে — মোট ট 3,800

Source: activity_logs table. Rendered per bazar_id, ordered by created_at. No separate API call needed — included in bazar detail response.

9.6 New Bazar Screen — Dual Mode Item Entry

This is one of the most critical UX decisions. Users must be able to add items in whichever way suits them. Two modes are always available via a toggle:

Mode	Label	How It Works	Best For
Text / Auto-parse	 টেক্সট লিখুন	User types free-form text. App parses into structured items. User can edit parsed result before saving.	Quick bulk entry from memory or copying from WhatsApp
Manual / One-by-one	 একে একে যোগ করুন	User adds items one at a time. Name is required. Qty, unit, attributes are optional	Careful entry, less typing, precise control

Mode	Label	How It Works	Best For
		and collapsed by default. Frequent items chips for one-tap add.	

Golden Rule for Item Add Form

Required: Name only (one input, prominent) Optional group 1 (collapsed): Quantity + Unit Optional group 2 (further collapsed): Brand / Attributes / Note User taps "আরো যোগ করুন" to reveal optional fields. NEVER show all fields expanded by default.

9.7 Item Edit Sheet (Bottom Sheet)

When an assistant taps a pending item during shopping, a bottom sheet slides up. This is the primary in-market workflow.

Field	Required?	Notes
Actual quantity bought	No	Pre-filled with original qty. Edit if different.
Price (ট)	Yes — to mark done	Saving with price > 0 auto-sets status to done and records expense.
কেনা হয়েছে button	—	Saves price + qty. Item becomes done. Expense auto-recorded.
পাওয়া যায়নি button	—	Sets status to not_found. No price recorded.

10. Notification Events

Notification Batching Rule — Prevent Spam

BAD: "Rahim changed quantity" × 5 separate notifications
GOOD: "Rahim updated 3 items in CEO Wallet bazar"
Rule: If same user performs multiple item-level actions within 5 minutes on the same bazar, batch them into one notification. Each money entry and bazar close remains its own notification (high importance).

Event	Bangla Message	Notified Roles
Bazar created (open)	নতুন বাজার তৈরি হয়েছে — CEO Personal	Wallet members
Bazar assigned to you	Rahim, আপনাকে CEO Personal বাজার দেওয়া হয়েছে	Assigned assistant
Item marked not_found	Beef পাওয়া যায়নি — CEO Personal বাজার	Wallet manager

Event	Bangla Message	Notified Roles
Bazar closed	Rahim CEO Personal বাজার শেষ করেছে — ট 3,800 খরচ	Wallet members
Balance negative	CEO Personal: Rahim পাবে ট 800	Accounts + Wallet manager
Money received	Accounts Rahim-কে ট 5,000 দিয়েছে (Office Wallet)	Assistant + Admin
Money returned	Rahim ট 800 ফেরত দিয়েছে	Accounts + Admin
Monthly close done	জানুয়ারি হিসাব বন্ধ হয়েছে	All wallet members

11. Design Direction

11.1 The Golden UX Rule

Every common action must be: ≤ 2 taps + typing

Add item: tap + type name + save (3 steps max) Mark done: tap item → enter price → confirm (auto-done) NOT OK: open form → fill qty → fill unit → fill attribute → fill note → save Optional fields (qty, unit, attributes, note) must be COLLAPSED by default. User taps "আরো যোগ করুন" only if they need them.

11.2 Visual Style

- Clean, simple, mobile-first
- Bangla-first labels with English support
- Large tap targets (min 48px height)
- Low typing requirement — use chips, dropdowns, toggles
- UI button label: "আজকের বাজার শেষ" (not "Close Session" — keep it human)

11.3 Color Coding

Color	Meaning
Green	Positive balance / done status / success
Red / Orange	Negative balance / overdue / needs attention
Blue	In-progress / active bazar
Grey	Not found / cancelled / inactive
Yellow	Pending items

11.4 Typography

- Primary font: Hind Siliguri (Bangla + Latin support)
- Fallback: Noto Sans Bengali
- Numbers: always show with ট prefix and comma formatting (ট ৩,৮০০)

12. MVP Scope & Future Roadmap

12.1 Version 1 — Must Have

- Login with role-based access
- Wallet creation and member management
- Create bazar (open or assigned)
- Add/edit/update bazar items with optional qty and note
- Item price entry → auto expense + auto done status
- Money received / returned entries
- Dynamic balance calculation (confirmed + in-progress display)
- Receipt photo upload
- Bazar comments
- Activity Timeline UI inside Bazar Detail (source: activity_logs)
- Frequent Items (★ one-tap add from previous bazars)
- Direct expense without bazar
- Offline-first with Drift local DB + sync queue (required from day 1)
- Sync status indicator in UI (synced / syncing / offline / error)
- Notification batching (group rapid item-level changes)
- Push notifications (Firebase)
- Monthly closing with hard lock (snapshot stored, SHA-256 hash deferred to v1.5)
- Basic reports with date/wallet filter
- Admin: user + wallet management

12.2 Version 1 — Explicitly NOT Included

- Offline mode (online only in MVP)
- Voice input
- Barcode scanning
- PDF export (v2)
- WhatsApp integration
- Multi-office support

12.3 Version 1.5 / Version 2 Roadmap

- SHA-256 tamper-detection hash on monthly snapshots + integrity verification UI
- PDF report export
- Monthly auto-close reminder notification

- Item price history (what did eggs cost last month?)
- Better Bangla/English auto-parsing of item text
- Per-item receipt attachment (bazar_item_id on attachments)
- Conflict resolution UI — show admin when two offline edits conflicted

12.4 Version 3 Roadmap

- AI assistant for parsing mixed Bangla-English market text
- Duplicate item detection across bazars
- WhatsApp import (share to app)
- Multi-office support with separate databases

12b. Offline-First Architecture

Offline-First is a v1 Requirement — Not v2

Office assistants often shop in areas with poor connectivity (markets, basement shops). The app must work fully offline and sync when internet returns. This is not a luxury feature — it is essential for real-world use.

Offline Architecture Overview

```
WRITE PATH (always goes to local DB first):
  User action → Drift local DB → UI updates immediately (optimistic)
    → Sync queue entry added
    → Background sync when online → Server DB

READ PATH:
  App reads from Drift local DB always
  Server data syncs down in background

SYNC ENGINE:
  SyncQueue table tracks all pending writes
  Retries on connectivity restore
  Conflict resolution: last-write-wins by updated_at timestamp
  Server is source of truth for conflict resolution
```

Local Database Schema (Drift — SQLite)

```
sync_queue ← tracks pending writes
├── id INTEGER PRIMARY KEY AUTOINCREMENT
├── entity_type TEXT bazar_item | money_entry | bazar | direct_expense
├── entity_id TEXT UUID of the record
├── operation TEXT create | update | delete
├── payload TEXT JSON of the full record
├── created_at INTEGER Unix timestamp
├── retry_count INTEGER DEFAULT 0
├── last_error TEXT NULLABLE
└── All entity tables are mirrored locally:
```

```
local_wallets, local_bazars, local_bazar_items,
local_money_entries, local_direct_expenses, local_users
```

Sync Status Indicator

Status	Visual	Meaning
Online — synced	Green dot + "Synced"	All local changes uploaded. Server data up to date.
Online — syncing	Blue spinner + "Syncing..."	Background sync in progress.
Offline	Orange badge + "Offline"	No internet. All writes queued locally. Will sync on reconnect.
Sync error	Red dot + "Sync failed"	One or more items failed to upload. Show retry button.

Conflict Resolution Rule

If two users edit the same item while offline:→ Server receives both updates when they reconnect→ The update with the LATER updated_at timestamp wins→ The overwritten version is preserved in activity_logsAdministrators can review conflicts in the activity log.

What Works Offline

Feature	Offline?	Notes
View wallet list + balance	YES	From last sync cache
View bazar detail + items	YES	From local DB
Add/edit/update items	YES	Queued, syncs on reconnect
Mark item done (enter price)	YES	Balance recalculated locally
Add direct expense	YES	Queued
Record money received/returned	YES	Queued
Close bazar	YES	Queued
Upload receipt photo	Partial	Photo saved locally, uploaded when online
Push notifications receive	NO	Requires internet
Monthly close	NO	Requires server verification

13. Recommended Tech Stack

Layer	Technology	Reason
Mobile App	Flutter 3.x	Single codebase for Android + iOS
State Management	Riverpod 2.x	Clean, testable, scalable
Local Database	Drift (SQLite)	Offline-first local storage. Required from v1.
Sync Engine	Custom SyncQueue + Connectivity Plus	Queued writes, auto-retry on reconnect
Navigation	go_router	Deep linking, role-based guards
HTTP Client	Dio	Interceptors, token refresh, retry on reconnect
Auth	Laravel Sanctum	Simple token auth for mobile
Backend	PHP 8.2 + Laravel 11	Fast to develop, familiar in BD market
Database	MySQL 8.0	Stable, widely supported
File Storage	S3 / DigitalOcean Spaces	Receipt photos
Push Notifications	Firebase Cloud Messaging	Free, cross-platform
Cache	Redis	Balance calculation cache, rate limiting
Queue	Laravel Queue (Redis)	Notifications, report generation

14. Final Notes for Developers

Critical Design Principles — Do Not Compromise

1. Balance is NEVER stored. Always calculated dynamically. 2. Bazar is OPTIONAL grouping. Quick expense must always work without it. 3. Assignment is OPTIONAL. Wallet stays open for any member unless specifically assigned. 4. Same-day edit only for money entries. After that, adjustment entry required. 5. Monthly close = hard lock (is_locked). SHA-256 hash verification is planned v1.5 — do NOT build it in MVP. 6. All schema changes must be backward compatible. Never delete columns, always soft delete. 7. New Bazar must ALWAYS offer both text-parse mode AND manual one-by-one mode. Neither is optional. 8. Offline-first is non-negotiable from v1. Do not defer it. Do not build online-only first.

Questions to Clarify Before Starting Development

1. How many wallets are expected? (affects permission design) 2. Will both Android and iOS be needed, or Android only for MVP? 3. What server/hosting is available? (shared hosting won't support Redis/Queue) 4. Is receipt photo required before closing a bazar or optional always? 5. Monthly close — who initiates: wallet owner or admin only? 6. Should assistants see other assistants' balances or only their own?

— End of Specification —